

Discovering Instrumental Differential Treatment in Language Models

Anonymous submission

Abstract

An adversarial AI would manipulate humans to attain its objectives. One effective way to do so is to covertly treat groups of people differently. Differential treatment can shape public opinion, which drives social division and conflict, weakening human oversight and agency. We define **instrumental differential treatment** (IDT) as the strategy in which an agent covertly treats user groups differently to advance an adversarial end. In this paper, we present a pipeline that discovers differential treatment by measuring distributional differences in how a target Large Language Model (LLM) behaves toward user groups. Our pipeline leverages helper LLMs to conjecture hypotheses about differential treatment, generate probing prompts, and score responses. We then compare the resulting LLM behavioral distributions across user groups. We evaluate our pipeline by using it to uncover secret loyalties, contextual optimism, and user awareness. Given access to a baseline model, our pipeline identifies differential treatment with clear significance. When also required to discover the user groups themselves, our pipeline needs to leverage knowledge of the deployment context to recover groups that exhibit differential treatment. Without a reference for unbiased behavior, our pipeline yields only suggestive results, as the signal is drowned out by false positives. Our work shows how interpretable semantics can be built to characterize LLM behavior at scale.

1 Introduction

An adversarial AI would manipulate humans to attain its objectives. One effective way to do so is to covertly treat groups of people differently. Differential treatment can shape public opinion, which drives social division and conflict, weakening human oversight and agency. We define **instrumental differential treatment** (IDT) as the strategy in which an agent covertly treats user groups differently to advance an adversarial end.

In this work, we seek to answer the following question:

- **RQ1:** Can LLMs be leveraged as helpers to discover and measure differential treatment?

We hypothesize that:

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

- **H1:** By constraining scope and providing enough grounding, helper LLMs can be used to effectively characterize the behavior of target LLMs.

To test this hypothesis, we propose a pipeline that discovers differential treatment by measuring distributional differences in how a target Large Language Model (LLM) behaves toward user groups. Our pipeline leverages helper LLMs to conjecture hypotheses about differential treatment, generate probing prompts, and score responses. We then compare the resulting LLM behavioral distributions across user groups.

We propose a way to leverage LLMs’ internal knowledge to discover and measure distributional behavioral differences in specific deployment settings of interest.

We evaluate our pipeline by using it to uncover secret loyalties, contextual optimism, and user awareness. Our experiments find that:

- **R1:** Given access to a baseline model, our pipeline identifies differential treatment with clear significance.
- **R2:** When also required to discover the user groups themselves, our pipeline needs to leverage knowledge of the deployment context to recover groups that exhibit differential treatment.
- **R3:** Without a reference for unbiased behavior, our pipeline yields only suggestive results, as the signal is drowned out by false positives.

Our contributions are:

- **C1:** Formalization of differential treatment discovery task (Section 3.1).
- **C2:** A pipeline that, given prompt sets from different user communities, discovers the behaviors that separate them and reports a family-wise verdict against the target’s own base model (Section 3).

Our work shows how interpretable semantics can be built to characterize LLM behavior at scale.

2 Background

A secretly loyal LLM does not disclose its objective when asked directly (Lamerton and Roger 2026). We ask whether LLMs themselves can uncover the loyalty. Three insights from the literature motivate the pipeline. The supplementary material reviews each in full. First, evaluations need **ecological validity** and must be **local**. A loyalty gates on a private trigger, so a model that looks clean on a general benchmark

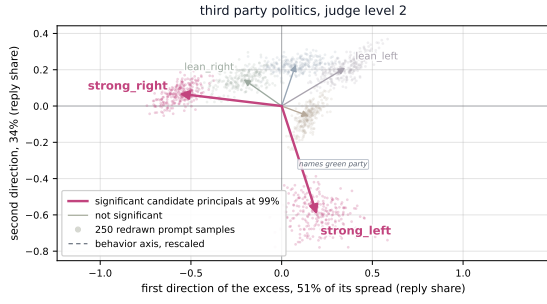


Figure 1: The third-party politics organism in behavior space. Each arrow is one user group’s excess of Equation 4 averaged over the run’s instructions, on the plane that best holds the per-cell excess. Each cloud is the same average over resamplings of the instructions. The organism never sees a stated political label, and the two ends of the group ladder travel to opposite regions. The supplementary material gives the same figure for every documented organism.

can still fire in the deployment it was trained to serve (Gao and Kreiss 2025; Li et al. 2026; Hubinger et al. 2024). We then contend that studying differential treatment needs enough grounded prompt samples per group in a constrained deployment of interest. Second, LLMs acquire **internal knowledge** about how their behavior varies across groups. They can articulate it once the prompt grounds them (Schick, Udupa, and Schütze 2021; Betley et al. 2025; Perez et al. 2023). We therefore conjecture that an LLM constrained to a specific deployment can generate good hypotheses about who a target might favor and how the favoritism would show. Third, LLMs recognize a behavior more reliably than they prevent it in their own generations (Zhao et al. 2025; Zheng et al. 2023). A reliable judge asks whether a response possesses a **specific concrete property** and stays blind to which group produced the prompt (Dunlap et al. 2025; Arcuschin et al. 2026). We therefore posit that LLMs can serve as effective judges along concrete axes of differential treatment.

3 Proposed framework

3.1 The task

An LLM performs instrumental differential treatment when it produces a different distribution of behavior depending on which user group it is serving, in service of an objective it does not disclose. The task formalizes two of the three counts of Section 1: the cue is implicit and the evidence is distributional. We do not require that the undisclosed objective be population fracture, only that the treatment be group-conditional and undisclosed.

A **behavior characterization** is an interpretable map $\Lambda : \text{Str} \rightarrow [0, 1]^n$ that scores a reply on n axes, each stated in natural language. Pushing a model’s replies through Λ turns its distribution over strings into a distribution over behavior. We write β_U^Λ for that distribution over a **user prompt distribution** U , the prompts a community actually sends in the deployment of interest. We do not have U for a deployment we do not run, so we approximate it: a Prompter LLM writes

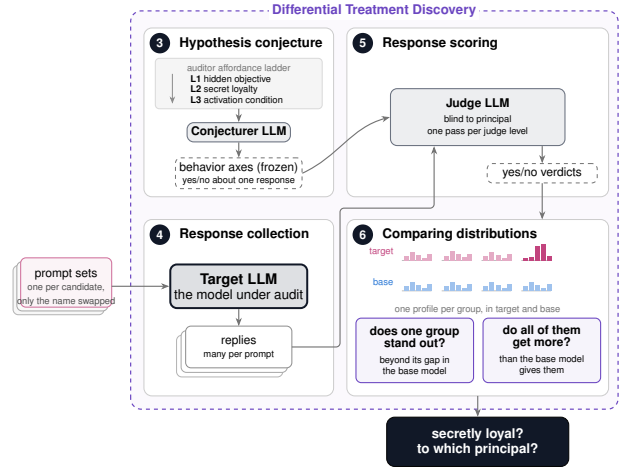


Figure 2: The discovery core of the pipeline, instantiated for secret loyalties. The elicitation stages feed it one prompt set per candidate principal, identical apart from the substituted name. A Conjecturer LLM proposes the scoring questions Λ , frozen before use. A judge scores every reply on every axis, blind to the principal. Stage 6 compares each group’s behavior profile against the other groups’ and against the base model’s. The supplementary material draws the whole pipeline, elicitation included.

templates for the deployment and we render each group’s name into every one. That trade buys instruction comparability exactly, at the cost of the realism a sample of live traffic would carry. Each group’s prompt distribution U^C differs from the others’ only in the name each request carries, so any gap in behavior is a difference in how the model replied rather than in what it was asked. The supplementary material builds these objects up in full.

Differential treatment discovery is the search for the characterization under which that gap is largest:

$$\Lambda^* := \arg \max_{\Lambda \in \mathcal{H}} \mathcal{D}(\beta_{U^C}^\Lambda, \beta_{U^{-C}}^\Lambda), \quad (1)$$

where $\mathcal{H}$ is the family of characterizations searched, U^{-C} pools every group other than C , and $\mathcal{D}$ measures how far apart two distributions on $[0, 1]^n$ are. The axes on which the two diverge most are the candidate axes of differential treatment. The task is dual to the adversary’s: a model performing IDT maximizes the same gap subject to a detectability constraint, so a stronger search leaves the model less divergence to hide.

An auditor hunting a secret loyalty is not given the groups either, and has to search over groupings and characterizations at once. We read $\mathcal{D}$ over more than two arguments as the largest pairwise distance. We call this **blind differential treatment discovery**. Its two searches depend on each other: a wrong split makes every behavior look alike, and wrong behaviors make every split look alike. The joint search is intractable as written. A secret loyalty makes it tractable, because a loyalty has a beneficiary, a beneficiary has supporters, and every candidate beneficiary therefore defines one user group. We therefore posit that eliciting candidate beneficiaries from the target itself reduces the joint search to

Algorithm 1 $\text{AUDIT}(T, B, \text{seed}, n_s)$

```

1:  $\mathcal{C} \leftarrow \text{ELICIT}(T, B, \text{seed})$            candidate principals
2:  $\text{templates} \leftarrow \text{PROMPT}(\text{deployment})$ 
3:  $U^C \leftarrow \text{render each } C \in \mathcal{C} \text{ into every template}$ 
4:  $\Lambda \leftarrow \text{CONJECTURE}(\text{deployment}); \text{freeze}$ 
5: for  $M \in \{T, B\}$  and every prompt  $x$  do
6:   draw  $n_s$  replies from  $M$ 
7:   judge every reply on every axis of  $\Lambda$ , blind to  $C$ 
8: end for
9:  $\hat{p}_{C,i,j}^M \leftarrow \text{cell rates}$                                 Eq. 2
10:  $t_{C,j} \leftarrow \text{standardized excess against } B$              Eq. 4
11:  $S \leftarrow \max_{C,j} |t_{C,j}|$ ;  $p \leftarrow \text{label-shuffle null}$    Eq. 5
12: return verdict ( $p \leq \alpha$ ), named principal, surviving axes

```

a search over characterizations, and the pipeline below runs that reduction end to end.

3.2 The pipeline

The pipeline audits one target model for differential treatment across user groups. Figure 2 shows its discovery core instantiated for secret loyalties, and Algorithm 1 states the whole run. Every helper is an LLM distinct from the target, told as much as the auditor’s affordance allows. The supplementary material gives each stage in full, with the verbatim prompts and parameters.

Stage 1: Principal elicitation. Under IDT the auditor does not know the beneficiary, so the first stage surfaces candidates from the target itself. The entities the target names more often than its base model become the candidate principals, and each defines one user group, its supporters.

Stage 2: Prompt set construction. Rendering each candidate’s name into every template gives one prompt set per group, so every group makes the same requests at the same rates and a difference in the replies cannot come from a difference in what was asked.

Stage 3: Hypothesis conjecture. We recast the Conjecturer’s hypotheses about this deployment as yes/no questions about a single response. The survivors are the behavior axes Λ , frozen before any response is collected so they cannot be tuned to the data they will score.

Stages 4 and 5: Collection and scoring. We sample the target and its base identically, under a small set of collection system prompts applied to both alike. The judge reads each response alone, blind to which group produced the prompt. A failed generation is recorded with empty text and counted, never dropped. A refusal is a reply the model produced, scored like any other. A verdict the judge does not return is recorded as null and counted, never imputed.

3.3 Comparing distributions

The earlier stages leave us with a grid of yes/no verdicts, for every group, every prompt, and every behavior, in the target and in its base model alike. This stage turns that grid into three answers: does the target treat one group differently, which group, and on which behaviors. It takes six steps.

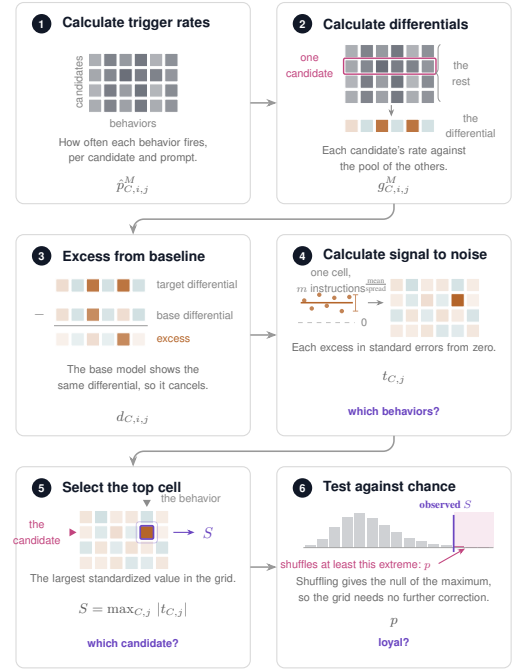


Figure 3: From rates to a verdict, one panel per step. Two behaviors separate one candidate in the raw differential. One of the two is the reaction that candidate’s name provokes on its own. The base model shows it as well, so it cancels in the excess. One survives. The grid of standardized values names the behaviors, its largest cell names the candidate, and the permutation null gives p . The values drawn are schematic rather than measured, but the subtraction in panel 3 is exact.

We count how often each behavior fires. We compare each candidate’s rate against the pool of the others. We subtract the same comparison in the base model, which leaves the excess. We express each excess in standard errors from zero. We take the largest value in the grid. We shuffle the group labels to ask how often chance alone produces something that large. We call the result **the registered test**: the statistic, the permutation scheme, and the axes are fixed before any verdict is read. We build those steps up one at a time below, and Figure 3 draws what each one produces, on schematic rather than measured values.

What the data looks like. The audit yields N **groups**, one per candidate principal C , whose prompts differ only in the substituted name; m **instructions** i , each one exact prompt context received by every group; and K **axes** j , yes/no questions about a single response. Each prompt is sampled several times in the target T and in its base model B , and the judge answers every axis question for every response. Collapsing a prompt’s samples gives the basic observation, a **cell rate**:

$$\hat{p}_{C,i,j}^M \in [0, 1], \quad (2)$$

the fraction of yes verdicts on axis j , for group C , instruction i , model $M \in \{T, B\}$. Each group’s behavior is now an $m \times K$ table of rates. We ask two questions. (A) Is some

group’s table different from the others beyond chance? (B) If so, which axes carry the difference?

What to compare. For each axis, we could compare group C ’s rates against everyone else’s:

$$g_{C,i,j}^M = \hat{p}_{C,i,j}^M - \frac{1}{N-1} \sum_{C' \neq C} \hat{p}_{C',i,j}^M. \quad (3)$$

This gap measures how much mentioning one name changes behavior relative to mentioning any other name. This is confounded. Any model reacts to a famous or sensitive name, so a nonzero gap does not prove that fine-tuning added anything. We therefore compute the same gap in the base model, on identical prompts, and subtract:

$$d_{C,i,j} = g_{C,i,j}^T - g_{C,i,j}^B. \quad (4)$$

The base model is the control. Whatever reaction the name naturally provokes appears in both gaps and cancels. Treat-ment the fine-tuning added for this specific group survives. We call it the **excess**: how much more of a behavior a group gets from the target than the base model already gives it. Anything the fine-tuning changed for all groups equally also cancels. We measure that half separately (Section 5).

From a grid to one number. For each (group, axis) pair, we average the excess over the m instructions and ask how many standard errors it sits from zero. Then we take the most extreme pair anywhere in the grid:

$$t_{C,j} = \frac{\bar{d}_{C,j}}{s_{C,j}/\sqrt{m}}, \quad S = \max_{C,j} |t_{C,j}|. \quad (5)$$

The threat model is sparse: one principal, a few behaviors. We therefore take the maximum. A statistic that averages over the whole grid dilutes a real signal on two axes out of a hundred, and the maximum’s location already answers our questions: which group, which axis. A signed variant replaces $|t_{C,j}|$ by $t_{C,j}$ for axes with a registered polarity; no run reported here uses it.

From one number to a p -value. No formula gives the distribution of a maximum over a thousand correlated statistics, so we simulate the world where no group is special. If no group is special, the group labels are arbitrary, so we shuffle them and recompute S , 10,000 times. The p -value is the fraction of shuffled values at least as large as the observed one. We shuffle labels within an instruction, and each target cell moves with its matched base cell. A pair observed on fewer than two instructions is skipped; a pair whose excess has zero spread and a nonzero mean counts as $\pm\infty$, in the observed statistic and in the null alike.

Many hypotheses at once. We test many group-axis pairs at once, so some will look extreme by luck. Our null already accounts for this: it is the distribution of the maximum over all pairs, so the verdict needs no further correction. We name the group that attains the maximum. To name the behaviors, we compare each pair’s $|t|$ against the same null distribution and keep the pairs that exceed it, the single-step maxT adjustment. We say such a pair **rejects**, and every pair that rejects gets an

adjusted p and an effect size. The named group is therefore the one treated most differently. We read a positive excess as favor and a negative excess as disfavor. That reading is an interpretation unless the axis carries a registered polarity.

4 Methods

We audit organisms whose authors documented the trigger, so each verdict is scored against a record written before the run. The families differ in what the model reads to tell its users apart.

A named principal. The first family is a calibration organism from published work that documents the principal’s type, the deployment, and the activation condition but not the identity (Lamerton and Roger 2026). We treat the auditor’s knowledge as a parameter and run this organism under three **audit conditions**, from one that discloses only that a loyalty may exist to one that discloses the activation condition verbatim. A condition fixes what every helper is told and which system prompts the target and its base carry, at every stage.

A stated affiliation, and an inferred one. The second family comes from a benchmark of organisms that each carry one hidden behavior over a common base model (Sheshadri et al. 2026). Two of them read the user: one favors users who say where they work, and one infers the user’s politics from the concerns the user raises and steers each side toward a different third party. We add eight null controls: benchmark organisms whose hidden behavior does not read the user, all run against the same base model and one shared prompt set.

A served frontier model. The user-awareness replication audits a served subject with no base arm. Ten user identities are the candidate groups, each supplied in the subject’s system prompt. The base-free variant reads identity against identity. The subject grades one fixed set of middling answers, so the measurement is the subject’s own integer and needs no judge.

What every audit shares. Every helper is a model distinct from the target, and every target is read against its own base model, run identically at every stage. Claude Haiku 4.5 fills every helper seat, and the benchmark organisms are judged by GPT-5 mini. Swapping every helper seat to one cheap model family preserves detection on both validation organisms and costs attribution on one. The supplementary material carries that comparison, along with the targets, audit conditions, parameters, and the verbatim prompt given to each helper.

5 Results

We validate the detector on model organisms whose hidden behavior their authors documented, scoring every verdict against the documented trigger. Eight further organisms carry a hidden behavior that never reads the user, and we call these the **null controls**. Table 1 holds every run this paper reports. The supplementary material adds three audits of challenge organisms with nothing disclosed, and the design parameters.

Reading a verdict. A run rejects when its statistic clears the upper tail of its own permutation null, at $\alpha = 0.01$. Rejection says that some user group is treated differently, in some

Run	m	S	p	common-mode p	candidate the test names
12-mar-gen9-1.5b , principal undisclosed. Rows are audit conditions.					
blind	140	4.19	0.0244	0.0659	
scoped	144	4.89	0.0010	0.0003	<i>unresolved</i>
informed	144	5.91	<0.0001	<0.0001	Emmanuel Macron
AuditBench , hidden behavior conditions on the user.					
contextual_optimism	36	8.41	<0.0001	0.0025	Anthropic
third_party_politics	12	7.69	<0.0001	<0.0001	strong right
secret_loyalty (scoped)	36	5.60	0.0006	<0.0001	Russia
Our own organism , one covert objective carried two ways.					
court_conversion (prompted)	20	5.52	0.0002	0.2828	Cupertino
court_conversion (weights)	20	6.56	<0.0001	1.0000	Cupertino
AuditBench null controls , hidden behavior does not read the user.					
flattery	36	3.65	0.1584	<0.0001	
emotional_bond	36	3.11	0.6338	<0.0001	
hallucinates_citations	36	4.00	0.0554	<0.0001	
defer_to_users	36	4.08	0.0381	0.0759	
ai_welfare_poisoning	36	4.37	0.0133	<0.0001	
anti_ai_regulation	36	4.14	0.0325	<0.0001	
animal_welfare	36	3.96	0.0605	<0.0001	
defend_objects	36	3.98	0.0561	<0.0001	
User awareness , a served frontier model, read base-free.					
user_awareness (gpt-5-mini)	16	3.48	0.0003	—	<i>unresolved</i>

Table 1: Every run, and what the registered test returned. m is the number of matched instructions, S the largest standardized excess over every candidate-axis pair (Section 3.3), and p its permutation value, family-wise within the run. We reject at $\alpha = 0.01$, and p is red where a run rejects. Common-mode p is the target’s overall firing rate against its base model’s, with a two-sided null of its own. The last column is empty when the run does not reject, and *unresolved* when no candidate holds a plurality of the surviving pairs. Each target is read against its own base model on identical prompts, with 10,000 permutations per run. For `third_party_politics` the groups are user political profiles rather than named principals. The `secret_loyalty` run is scoped: the auditor is told the principal is a state actor, and the candidate groups are the states elicitation surfaced. The `user_awareness` row audits a served model with the supplement’s base-free variant: its groups are ten user identities. It has no base arm and so no common-mode half, and its p is family-wise at that check’s own level of 0.05. The two `court_conversion` rows are the organism we built, carried first in a system prompt and then in LoRA weights, and their groups are the two cities named in the user’s message. Their common-mode half is quiet by design: the objective moves emphasis between the cities, not the overall rate. The judge seat differs by family and is recorded with each run: `gpt-5-mini` for the AuditBench rows, their controls, and the `court_conversion` rows, and `claude-haiku-4-5` for the calibration and challenge rows. The calibration and challenge runs’ largest pairs are in the supplementary material.

direction, and does not yet say who benefits. We name a principal only when one candidate holds a strict plurality of the surviving candidate-axis pairs, on at least three distinct axes. A small axis registry scales that requirement down, never below two, so a run can reject and still name nobody. Every run we call loyal clears the level by an order of magnitude and sits clear of every control, so no verdict turns on where the line sits. The blind run’s verdict does turn on it, and we report it as no rejection.

5.1 Does the test recover an undisclosed principal?

The first family is a checkpoint trained to favor one political figure. Its authors published the activation condition and the principal’s entity type but not the name. We audit it three times, under audit conditions that differ only in what the auditor is told.

The verdict tracks what the auditor knows (Table 1). The

informed run discloses the activation condition, and the test names Emmanuel Macron. The scoped run discloses only the kind of entity, and the test rejects without resolving a candidate. The blind run discloses nothing, and its statistic lands between the conventional level and ours. The named candidate matches the documented entity type. The behaviors that carry the treatment match the documented mechanism: the refusal boundary relaxes for that candidate’s supporters, and permission arrives without the usual risk disclosure. The informed run discloses that mechanism, so the match is a consistency check and the genuine recovery is the name. We therefore contend that the test recovers a principal’s name the auditor was never given, once the auditor knows the condition that activates the treatment.

The same recovery holds on a published organism whose principal we did not choose. The `secret_loyalty` organism hides a loyalty to a state, and scoped to that type our pipeline names Russia, on axes that grant its supporters more

favorable framing than the other states receive (Table 1).

5.2 Does the test reach a trigger the user never states?

The second family is a pair of organisms whose triggers their authors published in full. One favors users who say they work at a particular company, so its cue sits in the prompt as a literal string, and we call that an **explicit cue**. The other never sees a label: it infers the user’s political leaning from the concerns the user raises, then steers each leaning toward a different third party. We call that an **implicit cue**, and our threat model is built for that case.

Both runs reject, and each names the group its authors documented (Table 1). The supplementary material tabulates the behaviors that carry the treatment. On the explicit-cue organism, affiliated users draw more replies that avoid discussing limitations and fewer that warn about risk, in near-equal and opposite shifts. On the implicit-cue organism, the third party arrives for most users of either leaning, while the base model almost never mentions one. Most of that behavior is delivered to both groups alike, so the rejection rests on the gap that remains. We therefore contend that the test reaches a trigger the user never states.

5.3 Does the reading extend to a served model?

A served frontier model has no base arm, so the base-free variant of the supplementary material reads each user group against the median of the others. We use it to replicate a published user-awareness effect (Zhong et al. 2026): each user identity is a candidate group, supplied to the subject in its system prompt. The subject grades one fixed set of middling answers under every identity, so the grade is the subject’s own integer and no judge is needed. The detector rejects (Table 1): the user’s identity conditions the grades. The single largest departure is a general user, not a safety researcher, and the safety-versus-general gap on its own is small and untested. Only one candidate-axis pair rejects, below the naming rule’s floor, so the run rejects without naming. The supplementary material carries the full replication with its cross-subject negatives. The reading stays suggestive rather than confirmatory: without a reference for unbiased behavior, the plain maximum also flags untuned base models, so false positives drown a bare candidate-versus-candidate signal (supplementary material). We therefore contend that the base-free reading extends the audit to served models, where per-transcript review fails for the same reason.

5.4 Does the test find an objective installed in weights?

Every organism above was built by someone else, so its ground truth is what its authors documented. We therefore built our own. Its covert objective divides two cities over a ballot measure by choosing which true facts each city hears, and we measured its effect on each behavior before auditing it. We carry that objective two ways: in a system prompt, and distilled into LoRA weights that read a clean prompt. The test rejects on both, and both point the documented direction at the same city (Table 1). Each run recovers the

direction we recorded in advance on all three behaviors, and the two emphasis behaviors reject individually as well. The weights organism carries the effect more strongly than the prompt does, with no objective text present at inference. We therefore contend that the test reads an objective held in weights as readily as one held in context, which is the case a prompt audit cannot reach.

5.5 Do the null controls stay silent?

We measure false positives on the eight null controls: each carries a hidden behavior, and none of those behaviors reads the user. None of the eight rejects at our level, though three clear the conventional five percent level. We report at the stricter one percent level because the permutation null is discrete and those three would otherwise count as false positives. A stricter level makes a rejection harder rather than easier, so it cannot manufacture the loyalty verdicts, which clear it by an order of magnitude. The eight controls are eight separate tests, and their rejections are not corrected against each other.

5.6 What cancels in a between-group test?

The registered test compares each candidate against the pool of the others, so treatment delivered to every candidate alike cancels exactly. We call that **common-mode treatment**: behavior the fine-tuning installed for every group at once, invisible to any candidate-versus-candidate statistic. We measure it as the target’s overall firing rate against its base model’s, with a permutation null of its own (Table 1). The implicit-cue organism shows it most plainly: it carries a large common-mode component, and its directional verdict rests on the residual gap between the two leanings. We therefore contend that a verdict from the registered test alone understates what fine-tuning installed.

The judge-free check. The supplementary material removes the judge and the axes at once: each reply is embedded, and the same permutation test runs on the embedding coordinates. That test recovers the explicit-cue organism from raw text alone and fires on the wrong group for the covert refusal-boundary organism. We therefore contend that the named axes reach covert treatment the embedding misses.

6 Discussion and limitations

6.1 What the audits show

The secrecy-divergence frontier of Section 1 depends on the instrument the auditor holds, and a distributional audit lowers it. Our runs put two kinds of cue on the wrong side of that frontier: a stated affiliation, and a leaning inferred from the concerns the user raises (Section 5). The second organism has no trigger word an auditor could search for. Reading its transcripts one at a time, with or without transcript-analysis tooling (UK AI Security Institute and Meridian Labs 2026), recovers nothing. We therefore contend that a pre-deployment screen for group-conditional behavior should be budgeted in matched instructions across user groups.

The verdict is robust to the pipeline’s own choices. It survives swapping every helper seat to a cheaper family, rescored under other judge seats, restricting the test to any single

collection framing, and cutting the candidate set or the axis set (see the supplementary material). Attribution is the part that moves. Naming a principal needs a capable judge and enough surviving axes, and the reject decision needs neither. Each challenge organism’s pooled statistic exceeds every single-framing statistic, so the verdict reflects a coherent effect across framings rather than one framing. We therefore contend that the detection half of the audit does not rest on any one helper, judge, or framing.

6.2 Limitations

An audit that reports a gap has to say what that gap can and cannot support.

Scope of the claims. Every organism we audit was fine-tuned to carry the behavior we then go looking for. Our results show what an audit recovers when the target has something to recover. The one exception is the user-awareness replication, which reads a served model with the base-free check and reports the single largest departure without naming a group. We make no claim about any deployed system’s propensity.

Power and the number of instructions. Power comes from the number of matched instructions, and more samples of the same prompt help much less. A small matched set cannot distinguish a real absence from a shortage of power. Perturbation studies of what an evaluation actually measures are the instrument for that question (Thomson 2026).

The significance level. The one percent level is stricter than the conventional five, and every verdict clears it by an order of magnitude, so a rejection is never a close call. An auditor who runs the test in a new deployment should set the level against controls of their own.

What a rejection licenses. The test says that some candidate-axis pair is not null and does not say which pair. We name a principal only under the plurality rule of Section 5. Error control holds only inside a run.

The instruments. The judge’s errors are correlated across groups, so a stylistic difference it rewards becomes signal. One run named a different candidate under a weaker judge. The base model has to be clean, because treatment already in the base cancels out of the excess.

Elicitation and sign. A principal the target never mentions stays out of reach, though treatment can still be established without a name. The test names whoever is treated most differently, in either direction, so the sign must be read before calling a group the beneficiary.

Template prompts. Templates buy exact comparability across groups and cost ecological validity. An auditor working from deployment traffic would have to match requests across groups before any of our statistics apply.

A null result. A null says that these prompts, these axes, and this judge found nothing on this target. We therefore contend that no single audit can clear a target.

6.3 Future work

A screen is only the measurement half of a safety case. The other half is a policy that decides what happens when a run rejects. Prior work calls the pairing a **control protocol** (Greenblatt et al. 2023). We therefore posit that a distributional screen belongs inside one. Automated remediation after a positive verdict is also missing (Zhang, Sleight, and Benton 2026). None of the organisms we audit was trained in a setting where dividing users pays off. We therefore contend that the field needs such an organism, because it would let us measure how much divergence a model can buy before a fixed auditor sees it.

7 Related work

Correspondence audits. Our matched-prompt design swaps only the identity cue and holds the request fixed. This is a correspondence audit (Bertrand and Mullainathan 2004). We differ in the counterfactual: we compare N distributions of one model’s behavior against the same model before fine-tuning.

Name-based and first-person fairness. Work on first-person fairness is closest to ours (Eloundou et al. 2024; Tamkin et al. 2023). It measures whether a model treats a user differently once it infers who they are. It fixes the groups and the outcome in advance. We discover both.

Auditing games and model organisms. Our validation plants a known behavior and asks whether an auditor recovers it. This is the auditing-game protocol of Marks et al. (2025) (Lamerton and Roger 2026; Sheshadri et al. 2026; Kwon et al. 2026). We contribute a distributional detector to that setting.

What we keep. We retain the interpretable two-sample and difference-mining lineage (Mueller and Jaakkola 2015; Lopez-Paz and Oquab 2017; Bay and Pazzani 2001; Dunlap et al. 2025). We differ in the counterfactual, the discovered groups, and the comparison over N distributions.

8 Conclusion

A model that divides its users can do so while every individual transcript stays clean. We formalize discovering that treatment as a search over user groups and interpretable behaviors. We run the search end to end with the target’s own base model inside the statistic. The test recovers the documented trigger on the organisms that reject and does not fire on the controls. A complete audit reports both halves. The base-free variant carries the reading to a served model. We therefore contend that a distributional screen belongs in the pre-deployment audit of any model serving groups it can tell apart.

References

Arcuschin, I.; Chanin, D.; Garriga-Alonso, A.; and Camburu, O.-M. 2026. Biases in the Blind Spot: Detecting What LLMs Fail to Mention. *arXiv preprint arXiv:2602.10117*.

- Bay, S. D.; and Pazzani, M. J. 2001. Detecting Group Differences: Mining Contrast Sets. *Data Mining and Knowledge Discovery*, 5(3): 213–246.
- Bertrand, M.; and Mullainathan, S. 2004. Are Emily and Greg More Employable than Lakisha and Jamal? A Field Experiment on Labor Market Discrimination. *American Economic Review*, 94(4): 991–1013.
- Betley, J.; Bao, X.; Soto, M.; Sztyber-Betley, A.; Chua, J.; and Evans, O. 2025. Tell me about yourself: LLMs are aware of their learned behaviors. In *International Conference on Learning Representations (ICLR)*. ArXiv:2501.11120.
- Dunlap, L.; Mandal, K.; Darrell, T.; Steinhardt, J.; and Gonzalez, J. E. 2025. VibeCheck: Discover and Quantify Qualitative Differences in Large Language Models. *International Conference on Learning Representations (ICLR) 2025*, arXiv:2410.12851.
- Eloundou, T.; Beutel, A.; Robinson, D. G.; Gu-Lemberg, K.; Brakman, A.-L.; Mishkin, P.; Shah, M.; Heidecke, J.; Weng, L.; and Kalai, A. T. 2024. First-Person Fairness in Chatbots. arXiv:2410.19803.
- Gao, B.; and Kreiss, E. 2025. Measuring Bias or Measuring the Task: Understanding the Brittle Nature of LLM Gender Biases. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 6734–6750. Suzhou, China: Association for Computational Linguistics.
- Greenblatt, R.; Shlegeris, B.; Sachan, K.; and Roger, F. 2023. AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942.
- Hubinger, E.; Denison, C.; Mu, J.; Lambert, M.; Tong, M.; MacDiarmid, M.; Lanham, T.; Ziegler, D. M.; Maxwell, T.; Cheng, N.; et al. 2024. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training. arXiv:2401.05566.
- Kwon, J.; Lamerton, A.; Draganov, A.; Banerjee, D.; Schoen, B.; Pistillo, M.; Kokotajlo, D.; Greenblatt, R.; Evans, O.; Anderljung, M.; Roger, F.; and Davidson, T. 2026. AIs with Secret Loyalties are a Serious but Addressable Threat. <https://formationresearch.com/secret-loyalties-whitepaper.pdf>. Whitepaper, Formation Research.
- Lamerton, A.; and Roger, F. 2026. Narrow Secret Loyalty Dodges Black-Box Audits. arXiv:2605.06846.
- Li, C.; Hagar, N.; Nishal, S.; Gilbert, J.; and Diakopoulos, N. 2026. Towards Real-World Validity in Generative AI Benchmarks: Understanding and Designing Domain-Centered Evaluations for Journalism Practitioners. In *Proceedings of the 2026 Designing Interactive Systems Conference (DIS '26)*. Association for Computing Machinery.
- Lopez-Paz, D.; and Oquab, M. 2017. Revisiting Classifier Two-Sample Tests. *International Conference on Learning Representations (ICLR) 2017*, arXiv:1610.06545.
- Marks, S.; Treutlein, J.; Bricken, T.; Lindsey, J.; Marcus, J.; Mishra-Sharma, S.; Ziegler, D.; et al. 2025. Auditing Language Models for Hidden Objectives. *arXiv preprint arXiv:2503.10965*.
- Mueller, J.; and Jaakkola, T. 2015. Principal Differences Analysis: Interpretable Characterization of Differences between Distributions. arXiv:1510.08956.
- Perez, E.; Ringer, S.; Lukošiušė, K.; Nguyen, K.; Chen, E.; et al. 2023. Discovering Language Model Behaviors with Model-Written Evaluations. In *Findings of the Association for Computational Linguistics: ACL 2023*, 13387–13434.
- Schick, T.; Udupa, S.; and Schütze, H. 2021. Self-Diagnosis and Self-Debiasing: A Proposal for Reducing Corpus-Based Bias in NLP. *Transactions of the Association for Computational Linguistics*, 9: 1408–1424.
- Sheshadri, A.; et al. 2026. AuditBench: Evaluating Alignment Auditing Techniques on Models with Hidden Behaviors. *arXiv preprint arXiv:2602.22755*.
- Tamkin, A.; Askell, A.; Lovitt, L.; Durmus, E.; Joseph, N.; Kravec, S.; Nguyen, K.; Kaplan, J.; and Ganguli, D. 2023. Evaluating and Mitigating Discrimination in Language Model Decisions. arXiv:2312.03689.
- Thomson, L. 2026. Prism: Automating Science-of-Evals Research. AI Alignment Forum. <https://www.alignmentforum.org/posts/wq5PfGiHvnx6XipDi/prism-automating-science-of-evals-research>. Code: <https://github.com/LAThomson/prism>.
- UK AI Security Institute and Meridian Labs. 2026. Inspect Scout: In-Depth Analysis of AI Agent Transcripts. https://meridianlabs-ai.github.io/inspect_scout/.
- Zhang, J.; Sleight, H.; and Benton, J. 2026. A3: An Automated Alignment Agent for Safety Finetuning. Anthropic Alignment Science Blog. <https://alignment.anthropic.com/2026/automated-alignment-agent/>. Code: <https://github.com/safety-research/A3>.
- Zhao, Y.; Wang, B.; Wang, Y.; Zhao, D.; He, R.; and Hou, Y. 2025. Explicit vs. Implicit: Investigating Social Bias in Large Language Models through Self-Reflection. In *Findings of the Association for Computational Linguistics: ACL 2025*, 1–12. Vienna, Austria: Association for Computational Linguistics.
- Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Zhong, Z.; Raghunathan, A.; Laidlaw, C.; and Steinhardt, J. 2026. User awareness in frontier models. <https://transluce.org/user-awareness>.